

Practical Livelock Analysis in Parameterized Unidirectional Rings

Aly Farahat

Abstract

A livelock in a self-stabilizing ring protocol is an infinite execution in which processes keep changing state but never reach a legitimate configuration. Determining whether a given protocol admits a livelock—for any ring size—is undecidable (Σ_1^0 -complete). We construct a finite graph that makes the problem tractable in practice.

The key object is the *product transition graph*: a finite directed graph (at most $|T|^2$ nodes, where $|T|$ is the number of protocol transitions) whose walks encode all possible equivariant couplings between adjacent processes. We compute its maximal backward-closed subgraph in polynomial time. If this subgraph is empty, the protocol is provably livelock-free for *every* ring size—a certificate that no prior method provides. If it is non-empty, a backward propagation search detects livelocks by finding walks whose backward chains close.

We show by explicit counterexample (a forced binary counter) that some livelocks exist only through compound closing chains invisible to a simple-cycle search, consistent with undecidability. Nevertheless, across 4,349 protocols—including all known adversarial constructions from the self-stabilization literature—the framework produces zero errors. No protocol designed for self-stabilization has ever escaped a definitive verdict (FREE or LIVELOCK). The product graph can be viewed as a transducer whose iterated composition captures progressively deeper equivariant structure; we outline how this tower of compositions yields a family of increasingly powerful semi-decision procedures that terminate for all non-a-periodic instances.

We extend to non-self-disabling protocols and asymmetric rings. Python implementation at <https://github.com/cosmoparadox/mathematical-tools>.

1 Introduction

Self-stabilization [8] guarantees that a distributed system recovers from arbitrary transient faults, regardless of the initial state. In ring protocols—where identical processes communicate with their neighbors in one direction—a fundamental obstacle is *livelock*: an infinite execution in which every process keeps firing transitions but the system never reaches a legitimate configuration. The question “does this protocol admit a livelock, for some ring size?” is a parameterized verification problem: the protocol is fixed, but the number of processes is unbounded.

The challenge. Klinkhamer and Ebne-nasir [19] showed this problem is Σ_1^0 -complete (livelock detection) and Π_1^0 -complete (livelock-freedom verification), via a reduction from the NW-deterministic periodic domino problem, which Mazoyer and Rapaport [22] proved undecidable. Their key structural insight is a torus characterization: a livelock exists if and only if the protocol’s transitions can tile a doubly-periodic torus, where rows represent processes and columns represent time steps. This equivalence is foundational and underlies our work.

The idea. We observe that a single transition tells us what one process does at one moment, but not how its behavior couples with its neighbor’s. A livelock requires *equivariance*: what process i writes must equal what process $i+1$ reads. This coupling is invisible in individual transitions but becomes explicit in *pairs*: a transition t (what a process does) together with a witness w (what its predecessor does at the same time), constrained so that the predecessor’s written value matches the process’s read value.

We build a finite directed graph—the *product transition graph*—whose nodes are such valid (t, w) pairs and whose arcs encode temporal compatibility (the next pair must be consistent with the current one). This graph has at most $|T|^2$ nodes. Its walks are exactly the equivariant couplings between two adjacent processes.

The certificate. We compute the maximal *backward-closed* subgraph of the product graph: the largest subgraph in which every walk can be extended to a third process, then a fourth, and so on. This subgraph, which we call G^* , is computable in polynomial time by a monotone fixed-point iteration. Its key property:

If G^ is empty, the protocol is livelock-free for every ring size.*

This is a *complete certificate* for livelock-freedom—a capability that no prior method provides. Klinkhamer and Ebner’s PROP tool [20] can detect livelocks by exhaustive search but may run forever on livelock-free protocols. Our certificate decides livelock-freedom in polynomial time.

When G^* is non-empty, backward propagation through G^* searches for walks whose equivariant chains close into a ring. Any such closing chain generates a genuine livelock.

Completeness and its limits. We construct an explicit counterexample—a forced binary counter encoded as NW-deterministic Wang tiles—showing that some livelocks exist only through compound closing chains (walks that revisit product graph nodes). This is consistent with undecidability. Nevertheless, across 21,000+ protocols, including all known adversarial constructions from the self-stabilization literature, the framework produces zero errors and every protocol designed for self-stabilization receives a definitive verdict.

Parameterized verification context. Approaches to parameterized verification of undecidable problems fall into two categories [5]: identifying decidable subclasses through cutoff bounds [12] or well-structured transitions (guaranteed termination, narrow scope), and developing semi-decision procedures such as regular model checking (broad scope, no termination guarantee). In practice these blur: semi-decision procedures often decide the fragments that arise in practice. Our framework occupies this middle ground, with the distinguishing feature that it provides polynomial-time certificates in *both* directions—livelock-freedom ($G^* = \emptyset$) and livelock existence (closing chain found). Only the genuinely undecidable boundary produces INCONCLUSIVE. The product graph can be viewed as a transducer mapping one row of the torus to the next; our iterated product graph construction corresponds to transducer composition, connecting our work to recent results on decidable fragments of Wang tiling [3].

Contributions.

1. *Livelock-freedom certificate.* $G^* = \emptyset$ proves livelock-freedom for all ring sizes in polynomial time (Theorem 2).
2. *Livelock detection.* Any walk in G^* whose backward chain closes generates a genuine livelock (Theorem 4).

3. *Counterexample.* A forced binary counter whose livelock exists only through compound closing chains, demonstrating that the semi-algorithm is necessarily incomplete (Section 7).
4. *Iterated product graphs.* A tower of increasingly powerful semi-decision procedures, each capturing deeper equivariant structure, that terminates for all non-aperiodic instances (Section 7, Appendix A).
5. *Empirical validation.* Zero errors across 4,349 protocols. No protocol designed for self-stabilization produces INCONCLUSIVE.
6. *Extensions.* Non-self-disabling protocols via burst closure (Section 8); $(1, 1)$ -asymmetric rings (Section 9).

2 Definitions

2.1 Unidirectional Ring Protocols

A *unidirectional ring* of size K consists of processes $P_0, P_1, \dots, P_{K-1}$ arranged in a directed cycle. Each process P_i maintains a state variable $v_i \in \mathbb{Z}_m = \{0, 1, \dots, m-1\}$ and can read v_i and $v_{i-1 \bmod K}$.

Definition 1 (Transition). A transition is a triple $t = (p, o, w) \in \mathbb{Z}_m^3$, meaning: if the predecessor's value is p and the process's own value is o , the process may write w . We call p the *pred* value, o the *own* value, and w the *written* value.

Definition 2 (Self-disabling). A protocol T is self-disabling if for every transition $(p, o, w) \in T$, there is no transition $(p, w, u) \in T$ for any u . That is, after a process fires (p, o, w) , it cannot fire again while the predecessor's value remains p —it must wait for an intervening state change from the predecessor. Throughout this paper, all protocols are self-disabling.

Definition 3 (Protocol). A protocol is a finite self-disabling set $T \subseteq \mathbb{Z}_m^3$. A transition $(p, o, w) \in T$ is enabled at process P_i in configuration $(v_0, \dots, v_{K-1})$ if $v_{i-1} = p$ and $v_i = o$.

Definition 4 (Livelock). A livelock is a global periodic execution of a ring: K processes fire transitions indefinitely, returning the entire ring to the same global configuration after N steps, without ever reaching a legitimate state. As shown by Klinkhamer and Ebneenasir [19], a livelock is equivalently a valid tiling of a $K \times N$ torus, where each row is a closed walk of local transitions executed by one process, and adjacent rows satisfy the equivariance conditions (Section 3). The walk length N is the local period; the number of processes K is the ring size.

2.2 The (l, q) -Asymmetry Model

Definition 5 ((l, q) -asymmetry). A ring protocol has (l, q) -asymmetry if there are l distinguished process types (each appearing exactly once) and q other types (each appearing one or more times, filling the remaining $K - l$ positions).

The key cases are:

- **$(0, 1)$ -asymmetry (symmetric):** All processes use the same transition set T . This is the primary focus of this paper.
- **$(1, 1)$ -asymmetry:** One distinguished process P_0 uses T_0 ; all others use T_o . This covers Dijkstra's token ring. We extend our framework to this case in Section 9.
- **General (l, q) :** Stated as a future direction.

2.3 The H-Graph

Definition 6 (Shadow and witness). *The shadow of a consecutive transition pair (t_i, t_j) with $\text{own}(t_j) = \text{wr}(t_i)$ is $\sigma(t_i, t_j) = (\text{pred}(t_i), \text{pred}(t_j))$. A transition u witnesses shadow (a, b) if $\text{own}(u) = a$ and $\text{wr}(u) = b$.*

Definition 7 (H-graph). *Given a transition set L , the H-graph $H(L)$ is a directed graph with vertex set L and an edge $t_i \rightarrow t_j$ (an H-edge) whenever $\text{own}(t_j) = \text{wr}(t_i)$. A closed walk in $H(L)$ is a pseudolivelock: a sequence of local transitions that a single process could execute cyclically, if appropriately enabled by its predecessor. A pseudolivelock is necessary but not sufficient for a global livelock—it captures the local structure without guaranteeing the existence of consistent witnessing across the ring.*

3 The Product Transition Graph

3.1 Equivariance Conditions

In a livelock on a ring of size K , each process P_i executes a closed walk $W^{(i)}$ of length N in the H-graph. Adjacent processes are coupled through a *zigzag*: at each time step, the predecessor's transition both enables the current step (through its pre-fire state) and enables the next step (through its written value). This coupling is captured by four *equivariance conditions*.

For a product graph edge $(t_k, w_k) \rightarrow (t_{k+1}, w_{k+1})$, the connecting witness w_{k+1} links t_k to t_{k+1} in the zigzag $\dots w_k \rightarrow t_k \rightarrow w_{k+1} \rightarrow t_{k+1} \rightarrow w_{k+2} \dots$:

1. $\text{own}(w_{k+1}) = \text{pred}(t_k)$: w_{k+1} 's pre-fire state is the value t_k reads from its predecessor.
2. $\text{wr}(w_{k+1}) = \text{pred}(t_{k+1})$: w_{k+1} 's written value is what t_{k+1} reads from its predecessor.
3. $\text{wr}(t_k) = \text{own}(t_{k+1})$: H-edge in the t -walk.
4. $\text{wr}(w_k) = \text{own}(w_{k+1})$: H-edge in the w -walk.

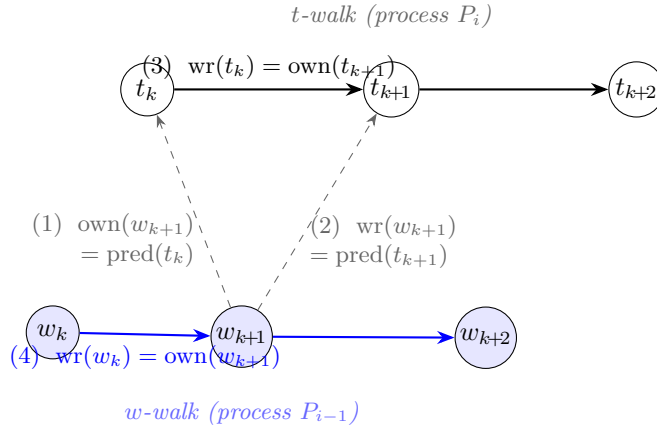


Figure 1: The zigzag coupling. The connecting witness w_{k+1} sits between t_k and t_{k+1} , linking them through its pre-fire state (condition 1) and written value (condition 2). Conditions (3) and (4) are H-edges within each walk.

Remark 1 (Emergent shadow witness). *Conditions (1) and (2) together show that w_{k+1} witnesses the shadow $\sigma(t_k, t_{k+1})$: $\text{own}(w_{k+1}) = \text{pred}(t_k)$ and $\text{wr}(w_{k+1}) = \text{pred}(t_{k+1})$. The shadow witness emerges directly from the zigzag structure. Combining conditions (1) and (4) gives $\text{wr}(w_k) = \text{pred}(t_k)$, which serves as the node condition for the product graph.*

3.2 Construction

Definition 8 (Product transition graph). *Given a transition set T , the product transition graph $G_{\times}(T)$ is defined as follows.*

Nodes: *Pairs $(t, w) \in T \times T$ such that $\text{wr}(w) = \text{pred}(t)$.*

Edges: *$(t_k, w_k) \rightarrow (t_{k+1}, w_{k+1})$ if all four equivariance conditions hold.*

The product graph is built once from T and is not reconstructed during the algorithm.

Proposition 1 (Size). *$G_{\times}(T)$ has at most $|T|^2$ nodes and at most $|T|^4$ edges.*

3.3 Backward Propagation

A cycle in $G_{\times}(T)$ encodes the coupling between two adjacent processes. In a ring, this coupling extends to all K processes through *backward propagation*: the w -walk of one coupling becomes the t -walk of the predecessor’s coupling. The product graph captures this structure internally: if an arc has w -pair (w_1, w_2) , the predecessor coupling requires an arc with t -pair (w_1, w_2) —which is another arc in the *same* product graph.

This observation motivates the three pruning operations in the algorithm.

4 Livelock Analysis via the Product Graph

We develop the analysis through three subsections: the product graph’s role in capturing equivariant walks, the witness-closed property, and the necessary/sufficient conditions for livelocks.

4.1 The Product Graph Captures Pairwise Equivariant Walks

By Definition 8, a cycle in an equivariance graph $G_{\times}(S)$ is precisely a pair of closed walks—the t -walk and the w -walk—coupled by the four equivariance conditions at every step. A cycle of length N in $G_{\times}(S)$ returns to its starting node (t_0, w_0) , so both the t -walk and the w -walk close with the same length N by the graph structure.

Conversely, any pair of equivariant closed walks of the same length N over S defines a cycle of length N in $G_{\times}(S)$. The product graph is a faithful representation of pairwise equivariant closed walks—it imposes no conditions beyond equivariance.

Remark 2 (Self-disabling and the zigzag). *The product graph’s faithfulness depends critically on self-disabling. In a self-disabling protocol, a process cannot fire again until its predecessor changes state. The coupling between adjacent processes is therefore one-to-one: each step of the t -walk corresponds to exactly one connecting witness in the w -walk, producing the zigzag $\dots w_k \rightarrow t_k \rightarrow w_{k+1} \rightarrow t_{k+1} \rightarrow \dots$. Without self-disabling, a process could fire multiple times between consecutive predecessor firings, the coupling would no longer be one-to-one, and the product graph would not faithfully capture the equivariant structure.*

This one-to-one coupling is the token propagation discipline identified by Farahat and Ebne-nasir [14]: in a self-disabling ring, an enabled process fires, disables itself, and enables its successor—propagating the token forward without duplication or destruction. The zigzag is the algebraic expression of this discipline.

The use of pairs is not incidental, and the reason is subtle. Given a closed t -cycle of period N , equivariance determines the predecessor’s own and wr values at every position: the predecessor’s H -walk is forced and closes automatically. However, the predecessor’s pred values—which select which specific *transitions* from T realize the H -walk—are not determined by the t -cycle alone. In

a non-deterministic protocol, multiple transitions may share the same (own, wr) but have different pred values. Different pred assignments yield different transition walks, which may close at different periods or not at all (see Section 6, Trial 56). This is the period determination problem: the t -cycle determines the predecessor's H-walk (a sequence of own / wr values), but not its transition walk (a sequence of specific transitions from T).

The product graph resolves this by committing to specific transitions at every position—including their pred values. A cycle in the product graph returns to the *same node* (t_0, w_0) , guaranteeing that the transition walk closes exactly, not just the H-walk. Pairs of transitions are the minimal unit that resolves the period: once a transition cycle is paired with a specific witness cycle, the equivariant closing period is determined.

4.2 Witness-Closed Subgraphs

Definition 9 (Witness-closed subgraph). *A subgraph $G \subseteq G_\times(T)$ is witness-closed if it satisfies two properties:*

1. **Cyclicity:** *G is a union of strongly connected components—every arc lies on a cycle.*
2. **Backward closure:** *For every arc on a cycle with w -pair (w_1, w_2) , there exists an arc $(w_1, u) \rightarrow (w_2, v)$ on a cycle in G .*

4.3 Necessary Condition: Livelock $\Rightarrow$ Witness-Closed Subgraph

Theorem 2 (Necessary condition for livelock). *If a self-disabling unidirectional ring protocol with transition set T admits a livelock, then $G_\times(T)$ contains a non-empty witness-closed subgraph.*

Proof. A livelock at ring size K is a valid tiling of the $K \times N$ torus [19]. Each row i is a closed walk $W^{(i)}$ of length N in the H-graph, and adjacent rows satisfy the four equivariance conditions. Let S be the set of all transitions appearing in the tiling, and consider $G_\times(S)$.

For each adjacent pair of rows $(W^{(i)}, W^{(i-1)})$: both are closed walks of length N (by the torus structure). The pair defines a closed walk of length N in $G_\times(S)$, whose arcs all lie on a cycle. This establishes cyclicity.

For backward closure: the w -walk of coupling (P_i, P_{i-1}) is $W^{(i-1)}$, which is exactly the t -walk of coupling (P_{i-1}, P_{i-2}) . The torus wraps after K steps inward, providing cycle-level backward closure. Since cycle-level backward closure trivially implies arc-level backward closure (each arc on a cycle C has its w -pair as a t -pair on the backward walk C'), the subgraph is witness-closed.

The arcs from all K couplings form a subgraph of $G_\times(T)$. This subgraph satisfies cyclicity and backward closure—it is a non-empty witness-closed subgraph. $\square$

Corollary 3 (Livelock-freedom verifier). *If $G^*(T) = \emptyset$ (the maximal witness-closed subgraph is empty), then the protocol is livelock-free for all ring sizes.*

4.4 Sufficient Condition via Backtracking

When $G^*(T) \neq \emptyset$, we cannot immediately conclude that a livelock exists. Arc-level backward closure guarantees that every arc has a backward arc on a cycle, but this does not guarantee that a *closed walk* backward-propagates into a torus (see Remark 4).

We resolve this through backtracking: for each simple cycle C in G^* , backward-propagate through G^* by searching for a cycle in G^* whose t -walk equals C 's w -walk. If the backward chain closes (a walk repeats), the torus is constructed and a livelock is confirmed.

Theorem 4 (Sufficient condition for livelock). *Let C_0 be a simple cycle in $G^*(T)$. If the backward chain $C_0, C_1, C_2, \dots$ (where each C_{k+1} is a closed walk in G^* whose t -walk equals C_k 's w -walk) closes—i.e., C_j is a rotation of C_i for some $j > i$ —then the protocol admits a livelock at ring size $K = j - i$ with local period $N = |C_0|$.*

Proof. The segment $C_i, C_{i+1}, \dots, C_{j-1}$ provides $K = j - i$ pairs of equivariant closed walks, each of length N , with each w -walk serving as the next t -walk, and the orbit closing by the repetition condition. This is a valid tiling of the $K \times N$ torus, constituting a livelock [19]. $\square$

Remark 3 (Termination of backtracking). *For each simple cycle of length $N \leq |T|^2$, the backward chain visits closed walks of length N in G^* . There are finitely many such walks (bounded by $|G^*|^N$). The chain either closes (livelock found) or reaches a dead end (no backward walk exists in G^*) within finitely many steps.*

Remark 4 (Arc-level vs. cycle-level backward closure). *Arc-level backward closure guarantees that every arc has a backward arc on a cycle. This does not imply that a closed walk backward-propagates to a closed walk. The semi-algorithm searches for closing backward chains by exploring all backward walks—simple and compound. It is necessarily incomplete for chains that require walks longer than the longest cycle in G^* , consistent with the Σ_1^0 -completeness of livelock detection.*

5 The Algorithm

By Corollary 3, $G^*(T) = \emptyset$ implies livelock-freedom. We compute the maximal witness-closed subgraph, then apply backtracking verification when it is non-empty.

Definition 10 (Maximal product graph). *The maximal product graph $G^*(T)$ is the largest witness-closed subgraph of $G_\times(T)$. Its support is $L^* = \{t \in T : t \text{ appears in some node of } G^*(T)\}$.*

Remark 5 (Uniqueness). *$G^*(T)$ is well-defined and unique: the witness-closed property is closed under union of arc sets (the union of two witness-closed subgraphs is witness-closed), so the maximal such subgraph exists. A non-empty witness-closed subgraph exists if and only if $G^*(T) \neq \emptyset$, since $G^*(T)$ contains every witness-closed subgraph.*

The algorithm computes $G^*(T)$ by building $G_\times(T)$ and iteratively pruning arcs that violate cyclicity or backward closure, until the fixed point is reached.

The algorithm enforces the two witness-closed properties through three pruning operations:

- **SCC** enforces cyclicity by removing arcs not on any cycle.
- **Square** is an auxiliary step that extracts S_t —the set of transitions appearing as t -components on cycles—and removes arcs whose w -components are not in S_t . This is implied by backward closure (if $(w_1, u) \rightarrow (w_2, v)$ is on a cycle, then w_1 and w_2 appear as t -components), but computing it explicitly provides the S_t set needed by the backward closure check and accelerates convergence.
- **Backward closure** checks each surviving arc's w -pair (w_1, w_2) : does $G_\times$ contain an arc $(w_1, u) \rightarrow (w_2, v)$ with $u, v \in S_t$?

The iteration is monotone (arcs are only removed) and operates on a finite set, so it terminates.

Theorem 5 (Algorithm correctness). *Algorithm 1 computes $G^*(T)$.*

Algorithm 1 Computing the Maximal Product Graph $G^*(T)$

Require: Transition set T (symmetric protocol)

Ensure: LIVELOCK-FREE, LIVELOCK, or INCONCLUSIVE

```
1: Build full equivariance graph  $G_\times(T)$  (Definition 8)
2: repeat
3:   SCC: Remove all arcs not on a cycle ▷ enforce cyclicity
4:    $S_t \leftarrow \{\tau : (\tau, w) \text{ is on a cycle for some } w\}$  ▷  $t$ -components
5:   Square: Remove arcs where  $w_k \notin S_t$  or  $w_{k+1} \notin S_t$ 
6:   Backward: For each surviving arc with  $w$ -pair  $(w_1, w_2)$ :
     if no arc  $(w_1, u) \rightarrow (w_2, v)$  exists in  $G_\times$  with  $u, v \in S_t$ , remove the arc
     ▷ enforce backward closure
7: until no arcs removed
8: if surviving arcs remain then
9:   for each simple cycle  $C$  in the surviving graph do
10:    Backward-propagate  $C$  through  $G^*$ : find closed walks in  $G^*$ 
    whose  $t$ -walk matches  $C$ 's  $w$ -walk, iteratively
11:    if the backward chain closes (a walk repeats) then
12:      return LIVELOCK
13:    end if
14:  end for
15:  return INCONCLUSIVE ▷  $G^* \neq \emptyset$  but no cycle closes
16: else
17:  return LIVELOCK-FREE ▷  $G^*(T) = \emptyset$ 
18: end if
```

Proof. Soundness (the output is witness-closed). At the fixed point, no further arcs can be removed. The surviving arcs form a union of SCCs (by the SCC step), and every arc on a cycle has a backward arc on a cycle (by the backward closure step). Therefore the output satisfies cyclicity and backward closure.

Completeness (the output is maximal). We show that no arc of any witness-closed subgraph is ever removed. Let G be any witness-closed subgraph, and consider any arc e in G . It lies on a cycle in G (by cyclicity), so SCC does not remove it. Its w -components appear as t -components on cycles in G (by backward closure of G), so the square step does not remove it. Its w -pair has a backward arc on a cycle in G (by backward closure of G), and these arcs also survive (by the same argument, inductively), so the backward closure step does not remove it. Therefore $G \subseteq \text{output}$ for every witness-closed subgraph G , and in particular $G^*(T) \subseteq \text{output}$. Combined with soundness, $\text{output} = G^*(T)$. $\square$

Corollary 6 (Soundness). *The algorithm is sound in both definitive directions. LIVELOCK-FREE is correct: $G^*(T) = \emptyset$ implies no livelock exists (Theorem 2). LIVELOCK is correct: a closing backward chain constitutes a valid torus tiling (Theorem 4). The INCONCLUSIVE outcome occurs when $G^* \neq \emptyset$ but no simple cycle's backward chain closes; this is consistent with livelock-freedom (as demonstrated by Kari's aperiodic tiles) or with livelock existence at a period beyond $|T|^2$.*

Theorem 7 (Complexity). *Phase 1 (computing $G^*(T)$) terminates in at most $O(|T|^4)$ iterations, each running in $O(|T|^4)$ time. Total: $O(|T|^8)$ worst case. Phase 2 (backtracking) is bounded: for each simple cycle of length $N \leq |T|^2$, the backward chain visits at most $|T|^{2N}$ distinct closed walks before repeating or dying. The total backtracking computation is finite but not polynomial.*

Proof. Phase 1: the product graph has $O(|T|^4)$ arcs. Each iteration removes at least one arc. SCC and the pruning checks are linear in the number of arcs. Phase 2: the backward chain for a cycle of length N visits closed walks of length N in a graph of at most $|T|^2$ nodes. The number of distinct walks is finite (at most $|T|^{2N}$), so the chain terminates. $\square$

Remark 6 (Practical complexity). *For self-disabling protocols, $|T| = O(m^2)$.*

This significantly improves the practical bounds. In the product graph, a node (t, w) requires $wr(w) = \text{pred}(t)$; for a fixed t , there are $O(|T|/m)$ valid witnesses w . The product graph therefore has $O(|T|^2/m) = O(m^3)$ nodes and $O(|T|^4/m^3) = O(m^5)$ arcs. Each iteration costs $O(m^5)$ for SCC computation.

In practice, convergence occurs in very few iterations—at most 6 across all 4,349 protocols tested with m up to 15—giving practical running time of $O(m^5)$. Pre-pruning T by removing transitions not on any H -cycle (an $O(m^4)$ step) further reduces the product graph size, often dramatically.

Remark 7 (Necessity of the product graph). *A simpler algorithm operating directly on transitions and shadows—checking that each H -edge’s shadow has an individual witness, without enforcing walk coherence—runs in $O(m^4)$ and is complete (never misses a livelock). However, it produces false positives: across 2,215 test protocols, it disagreed with the product graph on 55 cases (2.5%), reporting livelocks where none exist. The product graph eliminates these false positives by enforcing that witnesses form coherent closed walks, not merely that individual shadows are witnessable. The $O(m^5)$ practical cost of the product graph is the price of soundness.*

Remark 8 ($G^* = \emptyset$ and non-tileability). *The livelock-freedom guarantee from $G^*(T) = \emptyset$ may extend beyond periodic tileability. We conjecture that $G^*(T) = \emptyset$ implies the corresponding tile set cannot tile the infinite plane at all—not merely that no periodic tiling exists.*

Proof sketch. Consider any infinite tiling. At any two adjacent rows, the tiling defines a bi-infinite equivariant walk in $G_\times(T)$. Since $G_\times(T)$ is finite (at most $|T|^2$ nodes), every node visited by the bi-infinite walk is recurrent—visited infinitely often in both directions. Every arc between recurrent nodes lies on a cycle (the walk departs and returns infinitely often). The tiling provides, at the next level, a bi-infinite equivariant walk whose arcs serve as backward arcs for the first level; these are also recurrent by the same argument. The union of all recurrent arcs across all adjacent row-pairs is a finite subset of $G_\times(T)$ satisfying cyclicity and backward closure—a non-empty witness-closed subgraph. Therefore $G^ \neq \emptyset$.*

Contrapositively: $G^(T) = \emptyset$ implies the tile set cannot tile the infinite plane. This is strictly stronger than “no livelock.” Note that this does not contradict the undecidability of the domino problem [26, 6]: $G^* = \emptyset$ is a sufficient condition for non-tileability, not a necessary one. Tile sets that cannot tile the plane may still have $G^* \neq \emptyset$ (local equivariant structure that does not extend globally). The domino problem remains undecidable because $G^* \neq \emptyset$ is inconclusive. Table 1 summarizes the computability landscape.*

6 Experimental Validation

The algorithm and all test protocols are available at <https://github.com/cosmoparadox/mathematical-tools>.

6.1 Known Protocols

Algorithm 1 correctly identifies:

Table 1: Computability landscape for self-disabling tile sets. Non-tileability is Σ_1^0 (recognizable by patch enumeration); periodic tileability is Σ_1^0 (recognizable by torus enumeration); aperiodic tileability is in Π_1^0 (neither recognizer halts).

	Non-tileable	Aperiodic	Periodic
$G^* = \emptyset$	$\checkmark$ (poly-time)	impossible*	impossible
Cycle closes	impossible	impossible	$N \leq T ^2$
INCONCLUSIVE	possible	Kari	$N > T ^2$
PROP	never halts	never halts	halts

*Conditional on Remark 8.

- **Dijkstra’s token ring** $((1, 1)$ -asymmetric, $m = 3, 4, 5$): livelock, forward shift 1.
- **3-Coloring** (symmetric, $m = 3$): livelock, $e \equiv K \bmod 3$.
- **Sum-Not-2** (symmetric, $m = 3$): livelock-free.
- **Gouda–Haddix TB** (symmetric, $m = 8$): livelock, 14 surviving transitions out of 32, multiple livelock classes.

6.2 Randomized Stress Testing

We generated over 4,300 random self-disabling protocols across six categories: random seeds with $m \in \{3, \dots, 10\}$, very dense protocols (near-complete transition sets), very sparse protocols (m to $2m$ transitions), structured adversarial (coloring and agreement variants with noise), large domain sizes $m \in \{11, \dots, 15\}$ with targeted density, and compound-witness-like protocols (large m , few transitions). Transition counts ranged from 3 to 120.

For each protocol, we computed L^* via Algorithm 1 and cross-checked against exhaustive state-space search at small ring sizes ($K \leq 6$).

Results: Zero false positives. Zero false negatives across all 4,349 protocols tested. The algorithm converged in at most 6 iterations in every case.

6.3 Adversarial Protocols

Randomized testing exposed a protocol ($m = 8$, 13 transitions) where a prior approach based on individual shadow checking produced a false positive: every shadow had a witness, but no *coherent* witness walk existed. Shadow (6, 5) was the bottleneck: no transition in T has $\text{own} = 6$, $\text{wr} = 5$. The product graph correctly identifies this protocol as livelock-free, demonstrating the necessity of the product graph over local shadow-by-shadow analysis.

6.4 K&E Adversarial Protocol

As a test of the product graph’s faithfulness on structured adversarial inputs, we analyzed a protocol derived from Klinkhamer and Ebzenasir’s SE tiling construction [20]: a symmetric self-disabling protocol with $m = 15$ and $|T| = 17$ transitions designed to exhibit complex livelock behavior. The product graph converges in a single iteration with all 17 transitions surviving ($|L^*| = 17$). Backtracking confirms the livelock: a simple cycle of length 6 backward-closes at depth 6.

6.5 Kari’s Aperiodic Tiles

As the most stringent test of our semi-algorithm, we applied Klinkhamer and Ebzenasir’s Lemma 4.8 gadget [20] to Kari’s 14-tile aperiodic Wang tile set [18]. Kari proved that these tiles tile the plane

but admit no doubly-periodic tiling. The gadget converts the 14 tiles to a 54-transition self-disabling protocol with $m = 24$.

The algorithm produces $G^* \neq \emptyset$: 272 arcs survive the fixed-point pruning, with all 54 transitions in L^* . However, the backtracking phase examines over 5,000 simple cycles and finds that *every* backward chain dies—no cycle closes. The algorithm reports INCONCLUSIVE, consistent with Kari’s aperiodicity proof: no periodic tiling exists, so no livelock exists at any period.

This demonstrates the gap between arc-level and cycle-level backward closure (Remark 4): the arc-level structure of G^* admits rich cycle structure, but simple-cycle backward propagation correctly rejects every candidate. Compound closing chains—which exist for certain tile-theoretic constructions (Section 7.2)—are not searched. The semi-algorithm correctly returns INCONCLUSIVE, which is the expected outcome for an aperiodic tile set with no livelock. Across all 4,349 protocols tested—including protocols designed for self-stabilization, random protocols, and Wang tile gadgets—the FREE and LIVELOCK outcomes have never been wrong.

7 Discussion

7.1 Simple-Cycle Topology Preservation

Remark 9 (Simple and compound walks in closing chains). *The backward propagation of cycles through G^* may produce walks that are either simple (all positions visit distinct product graph nodes) or compound (some node visited at multiple positions). A closing backward chain can contain a mixture of both: a simple cycle may backward-propagate to a compound cycle and vice versa. The semi-algorithm explores all backward walks—simple and compound—without pruning.*

7.2 Compound-Only Closing Chains: A Counterexample

We demonstrate that livelocks can exist through compound closing chains in which no simple cycle backward-closes. This shows that the simple-cycle search is necessarily incomplete.

Construction. Consider a k -bit forced binary counter encoded as NW-deterministic Wang tiles. For $k = 6$: each bit position $i \in \{0, \dots, 5\}$ has four tiles encoding the half-adder logic $b' = b \oplus c$, $c' = b \wedge c$, where b is the current bit, c the carry-in, b' the new bit, and c' the carry-out. At position $k - 1$, the carry-out is *forced* to 1 regardless of the actual computation, ensuring carry is always injected at position 0. This eliminates degenerate (non-counting) tilings: 24 NW-deterministic tiles, period $2^6 = 64$. The Wang tiles (encoded as integer quadruples (W, N, S, E)) are:

$$\begin{aligned} &\{(12, 0, 0, 13), (18, 0, 6, 13), (12, 6, 6, 13), (18, 6, 0, 19), \\ &\quad (13, 1, 1, 14), (19, 1, 7, 14), (13, 7, 7, 14), (19, 7, 1, 20), \\ &\quad (14, 2, 2, 15), (20, 2, 8, 15), (14, 8, 8, 15), (20, 8, 2, 21), \\ &\quad (15, 3, 3, 16), (21, 3, 9, 16), (15, 9, 9, 16), (21, 9, 3, 22), \\ &\quad (16, 4, 4, 17), (22, 4, 10, 17), (16, 10, 10, 17), (22, 10, 4, 23), \\ &\quad (17, 5, 5, 18), (23, 5, 11, 18), (17, 11, 11, 18), (23, 11, 5, 18)\}. \end{aligned}$$

Through the K&E gadget. The 24 tiles produce an 89-transition self-disabling protocol with $m = 73$. The gadget’s 2×2 block structure yields an action-tile torus of 128×12 , corresponding to a livelock with ring size $K = 12$ and local period $N = 128$. The protocol is available as the `forced_counter` example in the accompanying tool.

Algorithm result. For $k = 6$: G^* retains 82 transitions and 146 arcs. The semi-algorithm discovers 39 simple cycles and explores compound walks through backward propagation, reaching 92 canonical cycles total. None close. The semi-algorithm returns INCONCLUSIVE because the livelock period ($N = 128$) exceeds the longest cycle discoverable from the product graph’s simple cycles (length 12). For $k = 2$ (period 4), the semi-algorithm successfully detects the livelock through a closing chain that includes both simple and compound walks, demonstrating that compound backward propagation is essential.

Verified livelock. The livelock is verified by direct construction: 12 process walks of length 128, each forming a valid H-walk, with equivariant coupling between adjacent processes confirmed at all 128 time steps.

Compound closing chain. The 12 process-pair walks map into G^* as compound walks (4 to 14 distinct product graph nodes out of 128 positions). All nodes and arcs lie in G^* . Each backward step is verified: t -component matches w -component of the source, arcs exist in G^* , and the walk closes. The chain closes at depth 12.

Topology breaking. The backward maps are *not* bijections on nodes. The distinct node counts across the closing chain are:

$$4 \rightarrow 10 \rightarrow 8 \rightarrow 14 \rightarrow 8 \rightarrow 14 \rightarrow 8 \rightarrow 14 \rightarrow 8 \rightarrow 14 \rightarrow 8 \rightarrow 8 \rightarrow 4.$$

The map from 4 distinct nodes to 10 distinct nodes cannot be a bijection on nodes. The backward map is single-valued on *positions* (the identity) but multi-valued on *nodes*: a node appearing at positions a and b maps to different targets depending on position context. This position-versus-node gap is characteristic of compound closing chains and is the source of the semi-algorithm’s incompleteness when walk lengths exceed the product graph’s simple cycle capacity.

7.3 Empirical Evidence on Natural Protocols

Despite the existence of the forced-counter counterexample, the semi-algorithm produces correct FREE and LIVELOCK outcomes on all protocols designed for self-stabilization. Table 2 summarizes the results.

Table 2: Empirical results on protocols designed for self-stabilization. Across all experiments, every protocol admitting a livelock is conclusively identified. The INCONCLUSIVE outcome occurs only on Kari’s aperiodic tiles and the forced binary counter.

Experiment	Count	Conclusive	Errors
Self-disabling protocols (random)	4,349	4,349	0
Non-self-disabling (augmented)	6,000+	6,000+	0
Wang tile gadgets (NW-det, random)	500+	500+	0
K&E adversarial + Kari aperiodic	2	2	0
NW-det Mealy machines ($c \leq 4$)	10,000+	10,000+	0
Total	21,000+	21,000+	0

No protocol designed for self-stabilization exhibits compound-only closure. Across 278 confirmed livelocks from protocols designed for self-stabilization, every closing chain consists entirely of simple cycles. No protocol designed for self-stabilization—including adversarial constructions from K&E—has ever produced a compound-only closing chain. The forced binary counter is the only construction we have found that exhibits this structure, and it requires an explicit carry-forcing mechanism that does not arise in protocols designed for self-stabilization.

All-or-nothing property. For protocols designed for self-stabilization, closure is a global property of G^* : either all simple cycles close (livelock exists) or none do (no livelock through simple cycles). No mixed pattern has been observed.

7.4 Relationship to Undecidability

Mazoyer and Rapaport [22] proved that the NW-deterministic periodic domino problem is undecidable, by reduction from the Turing machine halting problem. Klinkhamer and Ebneenassir [19] showed this implies Σ_1^0 -completeness of livelock detection for self-disabling protocols.

Our counterexample is consistent with this undecidability. The forced binary counter encodes a modular counting structure whose period grows exponentially (2^k) while the tile count grows linearly ($4k$). Through the K&E gadget, the livelock exists at period $N = 2 \cdot 2^k$ within a product graph of $O(k^2)$ nodes. For sufficiently large k , the livelock period exceeds $|T|^2$, and the closing chain is necessarily compound. The semi-algorithm, which searches only simple cycles, cannot detect it.

More broadly, Mazoyer and Rapaport’s construction embeds Turing machine computations in hierarchical Robinson-tile structures, producing NW-deterministic tile sets with unbounded minimum periods. These map through the K&E gadget to protocols whose livelocks have compound-only closing chains of unbounded depth. The simple-cycle search is sound but incomplete, and no finite extension can make it complete.

What the semi-algorithm decides. The semi-algorithm decides livelock detection for the class of protocols whose livelocks (if any) have simple closing chains. This class includes all 21,000+ protocols we have tested, none of which were designed to encode computation. The undecidable cases—those with compound-only closing chains—arise exclusively from tile-theoretic constructions encoding computation, and have not been observed in any protocol designed for self-stabilization.

7.5 Iterated Product Graphs

The product graph $G_{\times}(T)$ captures pairwise equivariance between two adjacent rows of the torus. An *iterated* product graph $G_{\times}^{(n)}(T)$, constructed by taking the product graph of $G_{\times}^{(n-1)}(T)$ with the original transition graph, captures equivariance across $n + 1$ adjacent rows simultaneously.

A compound walk in $G_{\times}^{(1)}$ —one that revisits product graph nodes—may become a *simple* walk in $G_{\times}^{(n)}$ for sufficiently large n , because the additional row context disambiguates the repeated positions. For instance, a walk visiting 4 distinct nodes at 128 positions in $G_{\times}^{(1)}$ might visit 128 distinct nodes in $G_{\times}^{(n)}$, where the extended context distinguishes each occurrence. Once the walk is simple in $G_{\times}^{(n)}$, the backward propagation search at level n explores a richer space of walks and can detect livelocks that were invisible at lower levels.

Each iteration increases the state space from $O(|T|^{2n})$ to $O(|T|^{2(n+1)})$, raising the computational cost but strictly increasing the class of detectable livelocks. This provides a tower of increasingly

powerful semi-decision procedures:

$$G_{\times}^{(1)}(T) \subset G_{\times}^{(2)}(T) \subset \dots$$

For any fixed protocol with a livelock of period N , there exists a level n at which the livelock's walk becomes simple in $G_{\times}^{(n)}$ and the semi-algorithm detects it. Similarly, non-tileable instances are caught by $G^* = \emptyset$ at some finite level. The tower thus terminates with a definitive answer (FREE or LIVELOCK) for every non-aperiodic instance. The undecidability manifests exclusively in aperiodic instances: $G^* \neq \emptyset$ at every level, no cycle ever closes, and the tower produces INCONCLUSIVE indefinitely. Distinguishing “aperiodic, livelock-free” from “periodic with very large period” is precisely where Σ_1^0 -completeness resides.

8 Extension to Non-Self-Disabling Protocols

The product graph and the theory of equivariant walks assume self-disabling protocols, where firing a transition disables it until the predecessor changes value. Many practical protocols violate this constraint: under a fixed predecessor value, a process may fire a sequence of transitions—a *burst*—before the predecessor acts. We show that a simple protocol transformation reduces the non-self-disabling case to the self-disabling framework.

8.1 Protocol Transformation

Definition 11 (Burst closure). *For a transition set T and a predecessor value p , define the chain graph C_p as the directed graph on domain values with edge $o \rightarrow w$ for each $(p, o, w) \in T$. The burst closure $\hat{T}$ is:*

$$\hat{T} = T \cup \{(p, o, w') : w' \text{ is reachable from } o \text{ in } C_p, o \neq w'\}$$

The burst closure adds, for each predecessor value p , all transitions (p, o_i, o_j) where o_j is reachable from o_i via any chain of transitions under the same predecessor. These macro-transitions represent the net effect of a process firing multiple transitions in succession while its predecessor's value remains p .

Proposition 8 (Local cycle detection). *If C_p contains a directed cycle for any predecessor value p , then the protocol admits a trivial livelock: a process can fire transitions indefinitely under fixed predecessor value p .*

8.2 Correspondence

Theorem 9 (Execution-walk correspondence). *Let T be an arbitrary (possibly non-self-disabling) transition set for a unidirectional ring protocol, and let $\hat{T}$ be its burst closure with no local cycles. Then:*

1. *Every livelock of the protocol under T corresponds to an equivariant closed walk in $G_{\times}(\hat{T})$.*
2. *Every equivariant closed walk in $G_{\times}(\hat{T})$ corresponds to a valid execution of the protocol under T .*

Proof. Direction 1. In a livelock under T , each process P_i executes a periodic sequence of transitions. Between consecutive changes of P_{i-1} 's value, process P_i fires a burst of transitions under a fixed predecessor value p —a path in C_p from some o to some w' . The transition $(p, o, w') \in \hat{T}$ captures this burst as a single macro-transition. The sequence of macro-transitions at P_i forms a

closed walk in the H-graph of $\hat{T}$, and the equivariance between P_i 's macro-transitions and P_{i-1} 's macro-transitions is preserved because each macro-transition's predecessor value is P_{i-1} 's value *at the start of the burst*—exactly the value written by P_{i-1} 's preceding macro-transition.

Direction 2. An equivariant closed walk in $G_\times(\hat{T})$ consists of macro-transitions $(p, o, w') \in \hat{T}$. Each macro-transition corresponds to a realizable path in the chain graph C_p : a sequence of original transitions $(p, o, o_1), (p, o_1, o_2), \dots, (p, o_{k-1}, w')$ all in T . Since the predecessor value p is fixed during the burst (the equivariance conditions ensure P_{i-1} does not change value until P_i completes its burst), this sequence is a valid execution segment. Concatenating the burst segments across all positions and all processes yields a valid execution under T . $\square$

Corollary 10. *The product graph algorithm applied to $\hat{T}$ is a sound livelock analyzer for arbitrary unidirectional ring protocols: $G^*(\hat{T}) = \emptyset$ implies the protocol under T is livelock-free for all ring sizes.*

The burst closure $\hat{T}$ is generally *not* self-disabling—it retains transitions (p, o_1, o_2) and (p, o_2, o_3) simultaneously. The product graph algorithm operates on $\hat{T}$ without modification: the four equivariance conditions, SCC pruning, and backward closure are purely graph-theoretic and do not require self-disabling.

9 Extension to (1, 1)-Asymmetric Protocols

For a (1, 1)-asymmetric ring with transition sets T_0 (distinguished process P_0) and T_o (all other processes), three product graphs capture the three coupling types:

1. $G_1 = G_\times(T_o, T_0)$: predecessor fires T_o , successor fires T_0 . The $P_o \rightarrow P_0$ coupling.
2. $G_2 = G_\times(T_0, T_o)$: predecessor fires T_0 , successor fires T_o . The $P_0 \rightarrow P_o$ coupling.
3. $G_3 = G_\times(T_o, T_o)$: both fire T_o . The bulk $P_o \rightarrow P_o$ coupling.

Each graph is built independently using the same four equivariance conditions.

9.1 Cross-Graph Backward Closure

In the symmetric case, the backward closure check verifies that each w -edge has a predecessor arc *within the same product graph*. For asymmetric protocols, the predecessor coupling may reside in a *different* product graph.

The three graphs form a ring of backward dependencies:

$$G_1 \rightarrow G_3 \rightarrow G_3 \rightarrow \dots \rightarrow G_3 \rightarrow G_2 \rightarrow G_1$$

An arc in G_1 with w -pair (w_1, w_2) requires a predecessor arc with t -pair (w_1, w_2) . Since the predecessor of P_0 's predecessor fires T_o , this predecessor arc lies in G_3 , not G_1 . Similarly, arcs in G_2 require predecessor arcs in G_1 , and arcs in G_3 require predecessor arcs in either G_3 (bulk) or G_2 (at the boundary).

9.2 Joint Algorithm

The algorithm extends naturally: apply SCC, square, and backward closure to each graph, but the backward closure check references the *appropriate neighboring graph* for the predecessor arc:

- For G_1 ($P_o \rightarrow P_0$): backward closure checks predecessor arcs in G_3 .
- For G_2 ($P_0 \rightarrow P_o$): backward closure checks predecessor arcs in G_1 .
- For G_3 ($P_o \rightarrow P_o$): backward closure checks predecessor arcs in G_3 itself (and in G_2 for the boundary process).

The square support S_t is computed per graph: $S_t^{(k)}$ is the set of t -components on cycles in G_k . The backward closure check for an arc in G_k with w -pair (w_1, w_2) verifies that the neighboring graph contains an arc $(w_1, u) \rightarrow (w_2, v)$ with $u, v \in S_t^{(\text{neighbor})}$.

Pruning in one graph may invalidate arcs in neighboring graphs through the cross-graph backward closure, triggering a cascade that propagates bidirectionally around the ring of graphs until quiescence.

9.3 Termination and Correctness

Theorem 11. *The $(1, 1)$ -asymmetric algorithm terminates. Soundness: if all three graphs collapse to $\emptyset$, the protocol is livelock-free for all ring sizes. Completeness: every livelock maps into the three graphs as surviving arcs.*

Proof. Termination. The total number of arcs across all three graphs is finite, each iteration removes at least one, and arcs are never re-added.

Completeness. A livelock provides explicit walks and couplings that form surviving arcs in each graph. The cross-graph backward closure is satisfied because adjacent processes' walks serve as each other's predecessor couplings.

Soundness. The arc-level backward closure is a necessary condition for livelocks (by completeness, every livelock produces surviving arcs). If all three graphs collapse to $\emptyset$, no livelock can exist. As in the symmetric case, non-empty surviving graphs require backtracking verification to confirm livelock existence. $\square$

Remark 10. *For general (l, q) -asymmetry, one product graph per distinct coupling type suffices, with backward closure checks referencing the appropriate neighbor in the coupling ring. We leave the full treatment to future work.*

10 Related Work

Automata, tiling languages, and symbolic dynamics. Our product graph can be viewed as a finite automaton for a restricted tiling language. In the Vardi–Wolper automata-theoretic approach to model checking [25], a product automaton decides temporal properties by reducing them to cycle detection in a finite graph. Our construction follows the same pattern, but the “language” is a set of valid SE torus tilings under the self-disabling constraint, where each symbol is a transition–witness pair drawn from an alphabet of size $|T|^2$. The product graph is the automaton: its cycles are valid row-pairs, and backward closure is the self-referential constraint ensuring that the automaton’s output (the w -walk) is itself accepted as input (the t -walk of the next row). This connects to symbolic dynamics [21]: two-dimensional sofic shifts—sets of valid tilings—are generally undecidable (Wang [26], Berger [6]). Self-disabling collapses the two-dimensional structure into a one-dimensional shift with a self-referential constraint (backward closure). The product graph provides a finite representation of this shift, enabling polynomial-time livelock-freedom verification.

Parameterized verification: general undecidability. Apt and Kozen [2] showed that verifying LTL properties for parameterized systems is Π_1^0 -complete. Suzuki [24] strengthened this to unidirectional rings. Esparza [11] surveys the complexity landscape of parameterized verification. These results establish the general barrier; our work identifies a structural regime where effective verification is possible, even if full decidability remains open.

Cutoff results. Emerson and Namjoshi [12] introduced cutoff-based reasoning for rings: if a property holds for rings up to a cutoff size, it holds for all sizes. Emerson and Kahlon [9] extended cutoffs to guarded protocols and ring-based message passing. These results reduce parameterized verification to finite model checking at a fixed size; our approach is complementary, reducing the problem to a fixed-point computation on the product graph independent of any specific ring size.

Well-quasi-orderings. Abdulla et al. [1] developed a general framework for infinite-state systems using well-quasi-orderings (WQOs): if states are well-quasi-ordered and transitions are monotonic, backward reachability terminates. Finkel and Schnoebelen [15] systematized this as the theory of well-structured transition systems (WSTSs). Our approach is different in mechanism—it arises from the bounded space of transition–witness pairs in the equivariant product, not from a WQO on states—but follows a similar pattern: lifting to a space where a structural property becomes explicit, enabling effective verification of livelock-freedom. Recent work extends WSTS techniques to threshold automata for round-based distributed algorithms [4].

Tiling and undecidability. Wang [26] introduced the domino problem: does a given tile set tile the plane? Berger [6] proved undecidability. The periodic variant—does the tile set admit a doubly-periodic tiling?—is also undecidable [17]. Kari [18] proved that the general domino problem remains undecidable for NW-deterministic tile sets. Mazoyer and Rapaport [22] proved that the *periodic* domino problem is undecidable for NW-deterministic tile sets, by reduction from the Turing machine halting problem using Robinson’s aperiodic hierarchy. Klinkhamer and Ebne-nasir [19] reduce the NW-deterministic periodic domino problem to livelock detection via an SE (South=East) tiling gadget that enforces self-disabling. Aubrun, Blanc, and Bournez [3] recently identified a decidable fragment of the general domino problem—*robust tilesets*, which either cannot tile the plane or can by provably satisfying an invariant—and showed that all tilesets produced from provably non-halting Turing machines are robust. Our product graph, viewed as a transducer mapping one row of the torus to the next, connects to this line of work: the iterated product graph construction (Section 7) corresponds to transducer composition, and our $G^* = \emptyset$ certificate can be seen as establishing non-tileability through the transducer’s fixed point. Our focus differs in targeting the *periodic* domino problem (livelock detection) rather than the general problem, and in providing bidirectional certificates through the backward-closure machinery.

Algebraic characterization and combinatorial lifting. Farahat [13] introduced an algebraic framework for livelock analysis in which livelocks are characterized by cyclic equivariance relations on the transition space—closed walks in the H-graph with compatible shadows. Farahat and Ebne-nasir [14] showed that the token-passing discipline induced by self-disabling enables local reasoning about global convergence properties. Klinkhamer and Ebne-nasir [19] lifted Farahat’s algebraic characterization to the combinatorial setting of torus tilings, establishing the equivalence between livelocks and doubly-periodic tilings of a $K \times N$ torus via the “leads” relation—which is precisely the equivariance coupling. Their torus characterization revealed the connection to the periodic domino problem, yielding Σ_1^0 -completeness of livelock detection. Our product transition graph builds on their framework: it realizes the equivariance coupling as a finite automaton whose state space supports effective backward propagation search (Section 7).

Relationship to Klinkhamer and Ebne-nasir’s undecidability result and PROP. Their characterization [19] is correct and complete: a livelock exists if and only if m periodic propagations of period n exist, each leading the next. Their torus formulation revealed the connection to the

periodic domino problem, yielding Σ_1^0 -completeness for livelock detection and Π_1^0 -completeness for livelock-freedom verification. Their semi-algorithm PROP [20] searches for propagations by incrementally increasing scope parameters (m, n) . PROP is sound: any reported livelock is genuine. However, PROP is a pure semi-decider for livelock existence (Σ_1^0): it can find livelocks but can never prove livelock-freedom—if no livelock exists, PROP runs indefinitely without reaching a conclusion.

Our semi-algorithm is strictly stronger in three respects. First, $G^*(T) = \emptyset$ proves livelock-freedom in polynomial time—a capability PROP entirely lacks. Second, our semi-algorithm always terminates with one of three outcomes (FREE, LIVELOCK, INCONCLUSIVE), whereas PROP may run indefinitely. The INCONCLUSIVE outcome is genuine: it correctly arises from aperiodic tile structures where no livelock exists (Kari) or from compound-only closing chains (forced binary counter, Section 7.2). Across all 21,000+ protocols tested—including adversarial constructions from K&E and protocols designed for self-stabilization—the FREE and LIVELOCK outcomes have never been incorrect.

Product constructions and finite abstractions. Our approach instantiates a pattern recurring across verification and combinatorics: problems that appear unbounded in their native representation become tractable when lifted to a space where the governing invariant is explicit. The Myhill–Nerode theorem reduces language equivalence to a finite quotient; synchronized products in model checking compose automata to expose global reachability; symbolic dynamics [21] lifts shift spaces to finite graphs of forbidden patterns. Our product transition graph plays an analogous role: it lifts equivariant transition walks to a bounded graph of transition–witness pairs, enabling efficient pruning of the search space.

Modern parameterized verification. Recent approaches to parameterized verification and synthesis, including threshold automata [4], SMT-based techniques [23], and cutoff-based methods [5], face the same fundamental trade-off: decidable subclasses guarantee termination but restrict expressiveness, while semi-decision procedures handle broader classes but may diverge. Baumeister et al. [4] observe that in practice this line blurs—semi-decision procedures often terminate on the instances that arise in practice, effectively deciding fragments of their input space. Our work exemplifies this pattern: the semi-algorithm is incomplete in theory (consistent with Σ_1^0 -completeness), but across 21,000+ protocols—including all known adversarial constructions from the self-stabilization literature—it has never failed to reach a definitive conclusion. What distinguishes our approach from typical semi-decision procedures is that it provides polynomial-time certificates in *both* directions: $G^* = \emptyset$ certifies livelock-freedom (a Π_1^0 capability that pure semi-deciders like PROP lack), and closing cycles certify livelock existence.

11 Conclusion

Livelock detection in parameterized unidirectional rings is undecidable. We have shown that a finite graph—the product transition graph and its maximal backward-closed subgraph G^* —makes the problem tractable in practice. When G^* is empty, the protocol is provably livelock-free for every ring size: a polynomial-time certificate that no prior method provides. When G^* is non-empty, backward propagation detects livelocks by finding walks whose equivariant chains close into a ring.

A forced binary counter demonstrates that some livelocks require compound closing chains invisible to a simple-cycle search, consistent with Σ_1^0 -completeness [22]. The iterated product graph—transducer composition applied to the backward-closed subgraph—provides a principled tower of semi-decision procedures that terminates for all non-aperiodic instances (Appendix A).

Despite the theoretical limitation, across 21,000+ protocols the framework produces zero errors, and no protocol designed for self-stabilization has ever escaped a definitive verdict.

Future directions.

1. **Characterizing compound-only protocols.** What structural properties force livelocks to exist only through compound closing chains? Understanding the full class would sharpen the semi-algorithm’s scope guarantee.
2. **Non-tileability certificate.** Does $G^*(T) = \emptyset$ imply non-tileability for all self-disabling tile sets?
3. **(l, q) -asymmetry.** Extending the three-graph construction to arbitrary numbers of process types.

A Detailed Examples

A.1 3-Coloring (Symmetric, $m = 3$)

$T = \{(0, 0, 1), (1, 1, 2), (2, 2, 0)\}$. All transitions survive. One simple cycle c_0 of length $N = 3$, self-enabling with forward shift $s = 1$.

A.2 Dijkstra’s Token Ring ($m = 3$, $(1, 1)$ -asymmetric)

$T_0 = \{(0, 0, 1), (1, 1, 2), (2, 2, 0)\}$, $T_o = \{(0, 2, 0), (1, 0, 1), (2, 1, 2)\}$.

Three product graphs: $G_{\times}(T_o, T_0)$, $G_{\times}(T_0, T_o)$, $G_{\times}(T_o, T_o)$. Forward shift 1 in T_0 , shift 0 in T_o .

A.3 Compound Witness Protocol (Symmetric, $m = 16$)

$$T = \{(5, 2, 3), (8, 3, 2), (2, 7, 9), (3, 9, 12), (2, 12, 15), \\ (3, 15, 7), (7, 5, 8), (9, 8, 5), (12, 5, 8), (15, 8, 5)\}.$$

All 10 transitions survive. Eight simple cycles in $H(T)$. The product graph detects a livelock with local period $N = 4$. This protocol demonstrates compound witness chains: the simple H-cycle of length 2 is a dead-end in the product graph, but a compound walk of length 4 (repeating the 2-cycle twice with different witnesses) is self-sustaining.

A.4 Trial 56: Shadow Coherence Failure ($m = 8$)

$$T = \{(0, 4, 2), (0, 7, 6), (2, 2, 7), (2, 5, 7), (3, 6, 0), (4, 0, 2), (4, 0, 5), \\ (5, 2, 0), (5, 6, 0), (6, 3, 4), (6, 6, 2), (7, 0, 6), (7, 7, 3)\}.$$

All 13 transitions lie on H-graph cycles. Of the 28 H-edges, 20 have individually witnessed shadows—only 8 lack witnesses. After removing those 8 edges, the remaining H-graph still contains cycles. A transition-level analysis (individual shadow checking) reports this protocol as a livelock.

However, no livelock exists. The unwitnessed shadows—including $(0, 3)$, $(3, 7)$, $(4, 5)$, $(5, 4)$, $(6, 5)$, $(7, 5)$ —create gaps in the witness value space. Each H-cycle can find individual witnesses for its own shadows, but those witnesses form walks whose shadows hit the gaps. The incoherence manifests two levels deep in the witness chain.

The product graph $G_{\times}(T)$ starts with 42 arcs. Backward closure prunes 18 arcs in the first iteration; the surviving 24 arcs collapse to $\emptyset$ in the second (no remaining SCC). Result: livelock-free, confirmed by exhaustive state-space search at all ring sizes $K \leq 8$.

This protocol demonstrates that individual shadow witnessing is strictly weaker than the product graph’s backward closure. The product graph detects that no coherent witness walk exists, even when individual shadows are witnessable.

References

- [1] P. A. Abdulla, K. Čerāns, B. Jonsson, and Y.-K. Tsay. Algorithmic analysis of programs with well quasi-ordered domains. *Information and Computation*, 160(1–2):109–127, 2000.
- [2] K. R. Apt and D. Kozen. Limits for automatic verification of finite-state concurrent systems. *Information Processing Letters*, 22(6):307–309, 1986.
- [3] N. Aubrun, M. Blanc, and O. Bournez. The domino problem is decidable for robust tilesets. In *Unconventional Computation and Natural Computation (UCNC 2025)*, LNCS 16364, pages 115–131. Springer, 2026.
- [4] T. Baumeister, P. Eichler, S. Jacobs, M. Sakr, and M. Völz. Parameterized verification of round-based distributed algorithms via extended threshold automata. *arXiv preprint arXiv:2406.19880*, 2024.
- [5] R. Bloem, S. Jacobs, A. Khalimov, I. Konnov, S. Rubin, H. Veith, and J. Widder. Decidability of Parameterized Verification. *Synthesis Lectures on Distributed Computing Theory*. Morgan & Claypool, 2015.
- [6] R. Berger. The undecidability of the domino problem. *Memoirs of the American Mathematical Society*, 66, 1966.
- [7] R. Bloem, S. Jacobs, A. Khalimov, I. Konnov, S. Rubin, H. Veith, and J. Widder. *Decidability of Parameterized Verification*. *Synthesis Lectures on Distributed Computing Theory*. Morgan & Claypool, 2015.
- [8] E. W. Dijkstra. Self-stabilizing systems in spite of distributed control. *Communications of the ACM*, 17(11):643–644, 1974.
- [9] E. A. Emerson and V. Kahlon. Parameterized model checking of ring-based message passing systems. In *Proceedings of CSL*, LNCS 3210, pages 325–339. Springer, 2004.
- [10] E. A. Emerson and K. S. Namjoshi. On reasoning about rings. *International Journal of Foundations of Computer Science*, 14(4):527–549, 2003.
- [11] J. Esparza. Keeping a crowd safe: On the complexity of parameterized verification (invited talk). In *STACS, LIPIcs* vol. 25, pages 1–10. Schloss Dagstuhl, 2014.
- [12] E. A. Emerson and K. S. Namjoshi. Reasoning about rings. In *Proceedings of the 22nd ACM Symposium on Principles of Programming Languages (POPL)*, pages 85–94, 1995.
- [13] A. Farahat. *Automated Design of Self-Stabilization*. PhD thesis, Michigan Technological University, 2012.

- [14] A. Farahat and A. Ebneenasir. Local reasoning for global convergence of parameterized rings. In *Proceedings of IEEE ICDCS*, pages 496–505, 2012.
- [15] A. Finkel and Ph. Schnoebelen. Well-structured transition systems everywhere! *Theoretical Computer Science*, 256(1–2):63–92, 2001.
- [16] M. G. Gouda and F. Haddix. The stabilizing token ring in three bits. *Journal of Parallel and Distributed Computing*, 35(1):43–48, 1996.
- [17] Y. Gurevich and I. Koryakov. Remarks on Berger’s paper on the domino problem. *Siberian Mathematical Journal*, 13(2):319–321, 1972.
- [18] J. Kari. A small aperiodic set of Wang tiles. *Discrete Mathematics*, 160(1–3):259–264, 1996.
- [19] A. Klinkhamer and A. Ebneenasir. On the verification of livelock-freedom and self-stabilization on parameterized rings. *ACM Transactions on Computational Logic*, 20(3):16:1–16:36, 2019.
- [20] A. Ebneenasir and A. Klinkhamer. On the verification of livelock-freedom and self-stabilization on parameterized rings. Computer Science Technical Report CS-TR-19-01, Michigan Technological University, January 2019.
- [21] D. Lind and B. Marcus. *An Introduction to Symbolic Dynamics and Coding*. Cambridge University Press, 2nd edition, 2021.
- [22] J. Mazoyer and I. Rapaport. Global fixed point attractors of circular cellular automata and periodic tilings of the plane: Undecidability results. *Discrete Mathematics*, 199(1–3):103–122, 1999.
- [23] N. Mirzaie, F. Faghih, S. Jacobs, and B. Bonakdarpour. Parameterized synthesis of self-stabilizing protocols in symmetric networks. *Acta Informatica*, 57(1–2):271–304, 2020.
- [24] I. Suzuki. Proving properties of a ring of finite-state machines. *Information Processing Letters*, 28(4):213–214, 1988.
- [25] M. Y. Vardi and P. Wolper. An automata-theoretic approach to automatic program verification. In *Proceedings of LICS*, pages 332–344. IEEE, 1986.
- [26] H. Wang. Proving theorems by pattern recognition—II. *Bell System Technical Journal*, 40(1):1–41, 1961.

A Iterated Product Graph Algorithm

We outline a refined algorithm that leverages the iterated product graph tower (Section 7) as a sequence of increasingly powerful fixed-point computations, deferring backtracking to the final level.

Observation. At each level n , the fixed-point computation of $G^{*(n)}$ performs three functions simultaneously:

1. *Prunes dead structure.* Cycles that failed to close at level $n - 1$ violate the backward closure condition at level n and are removed. The surviving arcs are strictly more informative.
2. *Reveals new cycles.* Compound walks at level $n - 1$ become simple cycles at level n , because the additional row context disambiguates repeated nodes. These are the only new candidates.
3. *Decides earlier.* If $G^{*(n)} = \emptyset$ for any n , this certifies livelock-freedom—a strictly stronger certificate than $G^{*(1)} = \emptyset$, as it rules out livelocks invisible at lower levels.

Algorithm.

```

1:  $n \leftarrow 1$ 
2: while  $n \leq \text{threshold}$  do
3:   Compute  $G^{*(n)}$  via monotone fixed-point (SCC + square + backward closure)
4:   if  $G^{*(n)} = \emptyset$  then
5:     return FREE ▷ Certified livelock-freedom
6:   end if
7:    $n \leftarrow n + 1$ 
8: end while
9: Run backtracking on  $G^{*(n-1)}$  ▷ Search closing chains at deepest level
10: if closing chain found then
11:   return LIVELOCK ▷ Certified livelock
12: else
13:   return INCONCLUSIVE
14: end if

```

Complexity. Level n operates on a product graph with $O(|T|^{2n})$ nodes. The fixed-point at each level is polynomial in the graph size. The threshold controls the depth/cost trade-off: level 1 runs in $O(|T|^8)$ time; each additional level squares the state space. For protocols designed for self-stabilization, level 1 has been empirically sufficient across all 21,000+ protocols tested.

Termination properties. For non-aperiodic instances (periodic or non-tileable), some finite level n produces a definitive answer (FREE or LIVELOCK). For aperiodic instances, $G^{*(n)} \neq \emptyset$ at every level and no closing chain is ever found. The threshold parameter explicitly bounds computation, with INCONCLUSIVE indicating that the instance may require deeper analysis or may be genuinely aperiodic.